Design and Evaluation of a Reinforcement-Learning Agent for Self-Healing Network Control: NeuroNet

A Complete Walkthrough of the Agent's Perception–Decision–Action–Learning Loop

Abstract

This paper documents the complete design and operation of the reinforcement-learning (RL) agent at the core of NeuroNet Modulator, a GPU-accelerated, self-healing network intelligence system. The work traces the agent's full lifecycle end to end: how raw network telemetry is transformed into a state representation, how an ensemble-gated Dueling Deep Q-Network (DQN) selects among eight discrete remediation actions, how a multi-component reward function scores the outcome of each action, and how experience replay and periodic retraining allow the agent to improve continuously from operational data. The agent is evaluated in a simulated Mininet environment across 20-, 50-, and 80-node topologies under three realistic failure scenarios — link congestion, routing instability, and link failure — achieving a median decision latency of 47-51 ms on GPU, an anomaly-detection accuracy above 89%, and an autonomous action rate of approximately 85% under a confidence-gated safety policy. The remaining 15% of decisions are escalated to a human operator, reflecting a deliberately staged, progressive-autonomy deployment model rather than a claim of unconditional autonomy. This paper is intended to serve as a citable technical description of the agent's architecture, training procedure, and empirical behaviour for academic reference.

Keywords: reinforcement learning, Dueling DQN, self-healing networks, GPU inference, experience replay, anomaly detection, network automation, safety-gated autonomy.

1. Introduction

Modern telecommunication and enterprise networks are managed largely through static, reactive mechanisms: fixed thresholds trigger alerts, and human operators diagnose and remediate faults after users have already experienced degraded service. Rule-based automation improves response time but cannot adapt to changing traffic patterns, and CPU-bound inference is too slow to act within the sub-second windows that modern service-level agreements (SLAs) require. NeuroNet Modulator addresses this gap by framing network fault remediation as a sequential decision-making problem and solving it with a GPU-accelerated reinforcement-learning agent that observes network state, selects a corrective action, and learns from the measured consequences of that action.

This paper is written as a self-contained technical account of that agent for use as a citable reference in academic work. Rather than describing the system as a whole, it focuses specifically on the reinforcement-learning component: how it is built, how it makes a decision, how it is trained from a cold start to steady-state operation, and how its behaviour was empirically validated. Section 2 formulates the problem as a Markov Decision Process. Section 3 describes state construction. Section 4 defines the action space.

Section 5 details the Dueling DQN architecture. Section 6 describes the confidence-gated safety mechanism. Section 7 defines the reward function. Section 8 covers the training strategy from synthetic pre-training through live adaptation. Section 9 presents the experimental evaluation. Section 10 discusses safety mechanisms and failure handling. Section 11 concludes and outlines limitations and future work.

2. Problem Formulation as a Markov Decision Process

Network fault remediation is formulated as a Markov Decision Process (MDP) defined by the tuple (S, A, P, R, γ), where S is the space of network states derived from telemetry and anomaly features, A is a discrete set of eight remediation actions, P is the (unknown) transition dynamics of the network under a given action, R is a designed multi-component reward function, and γ is a discount factor. The agent's objective is to learn a policy $\pi(a|s)$ that maximises expected cumulative reward, which in this domain corresponds to minimising latency, packet loss, and congestion while avoiding unnecessary or oscillating interventions.

The core architectural insight underlying this formulation is that network routing is structurally identical to a resource-allocation problem: an agent observes a live system state, evaluates a discrete set of possible interventions, and selects the one that maximises a measurable reward over time. This reframing is what allows a general-purpose deep RL architecture — originally designed for resource allocation (the SmartGPU architecture) — to be adapted to the networking domain with a redefined state, action space, and reward function.

3. State Representation

The agent does not observe raw telemetry directly. A perception pipeline first converts streaming metrics into a fixed-length state vector, which is the only input the RL policy ever sees. This section describes that pipeline end to end, from telemetry ingestion through anomaly scoring to final state construction.

3.1 Telemetry Ingestion

A Prometheus scraper polls each network device every 15 seconds and publishes readings through Kafka for exactly-once delivery, with InfluxDB retaining 30 days of history for offline replay and retraining. Five metrics are collected per node and link: round-trip latency to the next hop (ms), throughput relative to baseline (Gbps), per-flow packet loss ratio, buffer-utilisation-based congestion (normalised 0-1), and device health (CPU, memory, temperature).

3.2 Anomaly Detection and Feature Attribution

Because raw telemetry is noisy and baseline latency varies by link type, the agent does not act on absolute thresholds. Instead, an ensemble anomaly detector — a two-layer LSTM autoencoder (hidden dimension 64, latent dimension 16) combined with an Isolation Forest trained on 10,000 benign windows — scores each 15-second window for deviation from its learned normal pattern:

```
anomaly_score = 0.6 × lstm_reconstruction_error + 0.4 × isolation_forest_score
```

A score above 0.7 triggers the RL decision pipeline; scores between 0.5 and 0.7 are logged only, to suppress false positives; scores below 0.5 are treated as normal variation. When an anomaly is flagged, SHAP (SHapley Additive exPlanations) attribution identifies which input metrics drove the score — for example, a 40 ms latency spike accounting for 60% of the anomaly score and a 2% packet-loss increase accounting for 35% — and this attribution vector is passed directly into the RL agent's state, giving the policy an interpretable, feature-level view of why an anomaly was raised rather than an opaque scalar.

3.3 Final State Vector

The state consumed by the policy network concatenates the anomaly deltas, the SHAP attribution vector, a node identifier, the count of affected traffic flows, a cyclical (sine/cosine) encoding of time-of-day, and a short window of the five most recent actions taken at that node, so that the agent has memory of its own recent behaviour and can be penalised for oscillation. The final state dimensionality is 35, and depending on network size the underlying feature space from which it is derived spans 50-200 raw features.

```
state = {
  'anomaly_features':   [latency_delta, throughput_delta, loss_delta, congestion_delta],
  'shap_attribution':   [0.6, 0.2, 0.15, 0.05],
  'node_id':            network_node_identifier,
  'affected_flows':     count_of_affected_traffic_flows,
  'time_of_day':        hour_of_day_sin_cos_encoding,
  'recent_action_history': [last_5_actions],
}
state_dim = 35
```

4. Action Space

The agent selects from eight discrete, mutually exclusive actions per decision. Two of the eight are deliberately non-committal — escalation and monitoring — which gives the agent an explicit option to defer rather than being forced to intervene on every anomaly.

ID	Action	Effect	Execution Path
0	Reroute traffic (ECMP)	Distribute load across alternate paths	OpenFlow, 15-20 ms
1	Increase QoS priority	Prioritise affected flows in queue	Device, 5-10 ms
2	Scale up slice	Allocate additional 5G network-slice resources	Kubernetes, 40-60 ms
3	Reduce cross-slice interference	Throttle background slice traffic	Kubernetes, 20-30 ms
4	Adjust traffic shaping	Smooth out bursty traffic patterns	Device, 10-15 ms
5	Trigger local failover	Switch to backup link if available	Failover, 5 ms
6	Escalate to NOC + log	Human review (confidence < 75%)	Async, 0 ms
7	No action (monitor)	Collect more data before deciding	0 ms

Table 1. The eight-action discrete action space.

5. Agent Architecture: Dueling Deep Q-Network

The policy is implemented as a Dueling DQN, chosen over a standard DQN because separating the value of a state from the advantage of each action within that state improves convergence in the agent's relatively large, structured state space and produces a policy whose decisions are easier to interpret post hoc — a property that matters in a network-operations context where every autonomous action must be auditable.

```
Input (35 dims)
  → Dense(128, ReLU)
  → Dense(128, ReLU)
  → [Value stream]      Dense(1)   → scalar V(s)
  → [Advantage stream]  Dense(8)   → A(s,a) for each action
  → Q(s,a) = V(s) + (A(s,a) - mean(A(s,:)))
```

The two-headed output is recombined using the standard dueling aggregation, which subtracts the mean advantage to keep V(s) and A(s,a) identifiable. Training uses an experience-replay buffer (Redis-backed) holding the 5,000 most recent incidents, sampled in batches of 64 trajectories, with network updates triggered every 100 new incidents. The loss function is Huber loss, chosen for robustness to the reward outliers that occasionally arise from corrupted telemetry.

Policy exploration during learning follows an ε-greedy schedule, with ε decaying from 0.1 to 0.01 over the agent's first 10,000 live decisions, gradually shifting the agent from exploration toward exploitation of its learned policy as it accumulates operational experience.

6. Confidence Scoring and the Safety Gate

A central design decision is that no action is ever executed purely because it has the highest Q-value; it must also clear a confidence threshold. Confidence is estimated from disagreement across an ensemble of three independently trained DQN heads (same replay buffer, different random seeds). Low disagreement between heads implies high confidence; high disagreement — even when one head is confidently wrong — implies the agent should not act autonomously.

```
q_values   = model(state)                    # shape: [8]
q_ensemble = [model_1(state), model_2(state), model_3(state)]
q_std_dev  = std(q_ensemble)
confidence = 1.0 - (q_std_dev / (max(q_values) + epsilon))
```

In the MVP implementation, confidence is approximated with softmax probability from a single agent due to the single-agent training setup used for the working demo; the three-head ensemble formulation above is the full system design. The resulting confidence score is mapped to one of three operating regimes: at or above 0.75 the action executes autonomously and is logged; between 0.5 and 0.75 the action is recommended to the Network Operations Centre (NOC) for approval; below 0.5 the agent escalates and waits for explicit human sign-off before any action is taken.

6.1 Progressive Autonomy

The system does not claim full autonomy from initial deployment. It follows a staged three-phase model: Phase 1 (human-in-the-loop, 100% of decisions reviewed, active while the experience buffer holds fewer than 500 rows), Phase 2 (assisted automation, the phase evaluated in this paper, with high-confidence actions executing autonomously and an escalation rate of roughly 15%), and a target Phase 3 (full autonomy, escalation approaching 5% once confidence has been validated as reliable across 10,000+ live decisions). Escalation is treated as evidence the safety gate is functioning correctly, not as a system weakness.

7. Reward Function

Every executed action produces a scalar reward computed from the measured before/after change in four network metrics, an SLA-breach penalty, a per-action execution cost, and an oscillation penalty. Each component is capped to bound the influence of any single metric and the final reward is clipped to [-100, 100].

```
def compute_reward(state_before, action, state_after, sla_penalty=0):
    latency_delta    = state_before['latency'] - state_after['latency']
    latency_reward   = min(latency_delta * 2.0, 50)       # Cap at +50

    throughput_delta  = state_after['throughput'] - state_before['throughput']
    throughput_reward = min(throughput_delta * 0.5, 30)   # Cap at +30

    loss_delta   = state_before['packet_loss'] - state_after['packet_loss']
    loss_reward  = min(loss_delta * 10.0, 20)             # Cap at +20

    congestion_delta  = state_before['congestion'] - state_after['congestion']
    congestion_reward = min(congestion_delta * 5.0, 15)

    sla_reward = -sla_penalty * 100                       # -100 per min of breach

    action_cost = {0: -1, 1: -0.5, 2: -3, 3: -2, 4: -0.5, 5: -2, 6: -5, 7: -0.1}

    oscillation_penalty = -10 if action in state_before['recent_action_history'][-3:] else 0

    R = (
        0.50 * latency_reward    +
        0.20 * throughput_reward +
        0.15 * loss_reward       +
        0.10 * congestion_reward +
        1.00 * sla_reward        +
        0.05 * action_cost[action] +
        oscillation_penalty
    )
    return np.clip(R, -100, 100)
```

The component weights are configurable per SLA profile — for example, latency-sensitive real-time 5G video traffic weights latency more heavily than throughput, while batch data replication weights the reverse. Metrics that deviate more than three standard deviations from a 30-day rolling baseline are treated as corrupted telemetry and zeroed out of the reward calculation before it reaches the agent, preventing a single faulty sensor reading from corrupting the learned policy.

8. Training Strategy: From Cold Start to Steady State

Because a newly deployed agent has no operational experience, NeuroNet uses a four-phase training strategy that bootstraps the policy in simulation before allowing it to influence live decisions, and progressively relaxes constraints as the experience buffer grows.

Phase	Condition	Behaviour
1	Before live deployment	Synthetic pre-training on 5,000 Mininet simulation episodes
2	Buffer < 500 rows	Pre-trained weights active; confidence capped at 60%; all decisions NOC-reviewed
3	Buffer > 500 rows	Full RL operation; daily retraining on cumulative buffer; learning rate 0.0001
4	Production steady-state	Micro-batch retraining every 100 incidents; full retraining weekly

Table 2. The four-phase cold-start-to-steady-state training strategy.

8.1 Synthetic Pre-Training

Pre-training runs 5,000 episodes in a Mininet-simulated environment covering link congestion ramps (0-100% capacity), BGP-like route-flap routing instability, flow-table congestion and exhaustion, link failure and failover, gradually creeping device degradation, and combinations of multiple simultaneous failures. Across these 5,000 episodes, mean episode reward improves from approximately +5 to +45 — a roughly nine-fold improvement — and mean time to remediation (MTTR) falls from about 180 seconds under a rule-based baseline to about 120 seconds for the pre-trained RL agent. Qualitatively, the pre-trained agent learns to intervene early (rerouting when latency first rises by around 5 ms) rather than waiting for a large spike, which is the behavioural signature the reward shaping was designed to encourage.

```
for episode in range(5000):
    env = MininetSimulation(random_scenario())
    state = env.reset()
    while not env.done():
        action = dqn_agent.select_action(state)
        state_next, reward = env.step(action)
        replay_buffer.add((state, action, reward, state_next))
        if len(replay_buffer) > 64:
            batch = replay_buffer.sample(64)
            dqn_agent.train(batch)
        state = state_next
torch.save(dqn_agent.state_dict(), 'pretrained_dqn.pt')
```

8.2 Live Adaptation

Once deployed, the pre-trained weights seed live operation but confidence is deliberately capped at 60% and every decision is NOC-reviewed until the live experience buffer exceeds 500 rows (Phase 2). Beyond that point the confidence cap is lifted, and the agent retrains daily on its cumulative buffer at a reduced learning rate (0.0001) to avoid catastrophic forgetting of the pre-trained policy. At production steady-state, the agent retrains in micro-batches every 100 new incidents, with a full retraining pass weekly, keeping the policy current with drifting traffic patterns without requiring continuous full retraining.

9. Execution Layer

A selected action is not applied directly to the network by the RL module; it is dispatched through a separate execution layer that enforces the confidence gate and provides auditability and rollback. Every decision, whether autonomous or escalated, is logged to ElasticSearch before execution. Dispatch carries a 100 ms timeout — if the underlying system (OpenFlow controller, Kubernetes operator, or Ansible playbook, depending on the action type) does not confirm execution within that window, the action is marked as timed out and the NOC is alerted. For every action, a predefined inverse action is also prepared, and can be triggered automatically if the metrics measured after execution worsen within a 30-second follow-up window.

```
RL Decision (confidence, action_id, parameters)
    ↓
[Confidence ≥ 0.75?]
    ├─ YES → Execute immediately (async dispatch)
    └─ NO  → Queue for NOC approval
    ↓
[Action dispatch by type]
    ├─ Reroute (action 0)     → OpenFlow controller   → 15–20 ms
    ├─ QoS adjust (action 1)  → Ansible playbook       → 5–10 ms
    ├─ Scale slice (action 2) → Kubernetes operator    → 40–60 ms
    └─ ...
    ↓
[Action audit log → ElasticSearch]
```

10. Experimental Evaluation

All results below were obtained under controlled simulation conditions using Mininet, on an Intel Core i7-12700 CPU, an NVIDIA RTX 3060 GPU (12 GB VRAM), 16 GB RAM, and Ubuntu 22.04. All latency values are empirically measured under controlled workload conditions rather than derived analytically.

10.1 Scaling Validation

To confirm that decision latency and detection accuracy remain stable as the managed network grows, the agent and reward function were held fixed while only the topology definition was changed across a 20-node linear chain and 50- and 80-node fat-tree topologies.

Topology	Nodes	p50 Latency	p95 Latency	Detection Accuracy	Avg Recovery Time
Chain	20	~47 ms	~52 ms	~90-92%	~48 ms
Fat-tree	50	~49 ms	~55 ms	~90-91%	~51 ms
Fat-tree	80	~51 ms	~58 ms	~89-91%	~54 ms

Table 3. Latency and detection accuracy across scaled topologies.

Decision latency increases by only about 4 ms (8.5%) as the topology scales four-fold from 20 to 80 nodes, and detection accuracy stays above 90% across all sizes, indicating that the LSTM autoencoder generalises across network scale without requiring retraining for each topology.

10.2 Performance Benchmarks

Metric	Value	Condition
p50 inference latency	~47 ms	20-node chain, GPU inference
p95 inference latency	~52 ms	20-node chain, GPU inference
p50 latency (80-node)	~51 ms	80-node fat-tree, GPU inference
p95 latency (80-node)	~58 ms	80-node fat-tree, GPU inference
Decision rate	~12 decisions/sec (bursts)	100 episodes, 3 anomaly types
Anomaly detection accuracy	~89-92%	LSTM + Isolation Forest ensemble
Autonomous action rate	~85%	Confidence ≥ 0.75 threshold
Escalation rate	~15%	Confidence < 0.75 threshold

Table 4. GPU inference and decision-quality benchmarks.

10.3 Failure Scenario Evaluation

Three scenario families, each approximating a real telecom disruption rather than a synthetic traffic spike, were run for 33-34 episodes per topology size.

Scenario	Action Taken	Recovery Outcome	Mean Reward
Congestion (95% link utilisation)	Reroute traffic (ECMP)	Latency ↓ 18.3 ms, loss ↓ 1.4%	+38.5 (σ=9.2)
Routing instability (BGP flap)	QoS adjust + traffic shaping	Latency ↓ 14.7 ms, stability restored	+32.1 (σ=11.4)
Link failure (throughput → 2 Gbps)	Trigger local failover	Latency ↓ 22.1 ms, throughput ↑ 8.1 Gbps	+41.2 (σ=8.1)

Table 5. Outcomes across the three evaluated failure scenarios.

In every scenario family, the anomaly was detected within a single 15-second telemetry scrape cycle, and the corrective action produced a positive mean reward with a bounded standard deviation, indicating consistent rather than occasionally-lucky remediation behaviour.

10.4 Extended Stability Run

A 300-episode sustained run (approximately six hours of equivalent network activity, 100 injected anomalies per scenario type) was used to check for policy drift, crashes, or reward-signal corruption over time. The system completed the run with zero critical failures, a stable mean decision latency of 49 ms showing no drift, and zero reward-signal corruptions (all components remained within the 3σ baseline). Three oscillation events were correctly identified and penalised by the reward function, each resolving within two subsequent episodes.

11. Safety Mechanisms and Failure Handling

Six safeguards operate continuously alongside the learning agent, summarised in Table 6.

Mechanism	Description
Confidence gating	Actions execute only when confidence ≥ 75%; lower-confidence decisions escalate to the NOC.
Reward signal validation	Metrics deviating beyond 3σ from a 30-day baseline are treated as corrupted and zeroed out.
Rollback capability	Every action has a predefined inverse action to restore system state if performance degrades post-execution.
Oscillation prevention	Repeated actions within a short window are penalised at −10 reward to discourage thrashing.
Execution timeout	Actions exceeding a 100 ms execution window trigger alerts and fallback mechanisms.
Audit log	Every decision, autonomous or escalated, is logged to ElasticSearch before execution.

Table 6. Safety mechanisms active during the evaluated MVP.

It is important to distinguish where the sub-50 ms latency figure applies: it refers specifically to RL inference and decision-making time on GPU, not the full response chain. The complete end-to-end path includes the 10-15 second telemetry collection interval, the sub-50 ms GPU inference step, and a further 5-60 ms of action execution latency depending on action type. The RL agent's contribution is to make the decision step itself effectively instantaneous relative to human-driven remediation (which typically takes 3-5 minutes), not to eliminate the telemetry sampling interval that precedes it.

12. Conclusion, Limitations, and Future Work

This paper has traced the NeuroNet Modulator reinforcement-learning agent through its complete lifecycle: telemetry ingestion and anomaly-driven state construction, an eight-action discrete action space, a Dueling DQN policy with ensemble-based confidence gating, a multi-component reward function that balances latency, throughput, loss, congestion, SLA penalties, action cost, and oscillation avoidance, a four-phase cold-start-to-steady-state training strategy, and an execution layer that enforces safety, auditability, and rollback. Evaluated in simulation across 20-, 50-, and 80-node topologies and three realistic failure scenarios, the agent sustained sub-52 ms p95 decision latency, above-89% anomaly-detection accuracy, and an approximately 85% autonomous action rate with the remaining decisions safely escalated to human review.

Several limitations should be acknowledged for academic completeness. First, all results reported here are simulation-based (Mininet); the agent has not been validated against a production network, and real-world traffic, hardware heterogeneity, and adversarial conditions may behave differently from the simulated topologies. Second, the MVP approximates ensemble confidence with a single-agent softmax rather than the full three-head ensemble described in the architecture, so reported confidence and

escalation figures reflect that simplification. Third, the reward function's fixed component weights, while configurable per SLA profile, were not exhaustively tuned or ablated in this study. Future work includes production pilot deployment, a full three-head ensemble implementation, systematic reward-weight ablation, and extension of the state and action spaces to Mobile Ad Hoc Network (MANET) settings, for which the underlying RL formulation is designed to generalise without structural change to the reward function.